Relatório de Auditoria de Segurança — A Bancada Evangélica

Isolamento de tenant, permissões, IDOR, segredos e inputs sem tratamento

DATA
2026-09-08

REPOSITÓRIO
github.com/rilsonjoas/a-bancada-evangelica

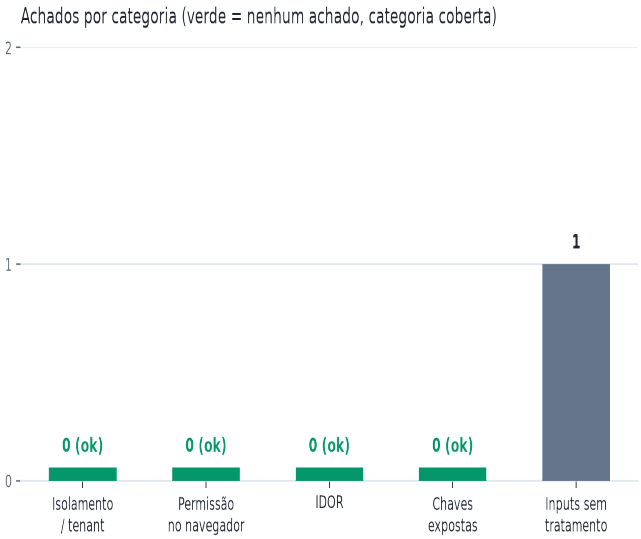
ESCOPO AUDITADO
Repositório a-bancada-evangelica: frontend React/Vite (src/pages, src/components), API NestJS + Prisma/Postgres (src/api, 8 módulos: politicians, votes, parties, stats, methodology, contact, news, health), serviço Python FastAPI de clustering/ML (analysis/, psycopg2 + scikit-learn) e scripts de sincronização/seed que rodam via cron (sync-worker.ts). Escopo: todas as rotas dos dois backends, os DTOs de query, o guard de admin (news), o formulário de contato, as queries SQL brutas do serviço Python, config/env dos três subprojetos, e a superfície de renderização do frontend.

NOTA METODOLÓGICA
Mecanismo de isolamento equivalente a RLS: **não há conta de usuário final** — é uma plataforma de transparência com dado público por design (votos, gastos, scores de parlamentares). A fronteira de autorização real é o **status de curadoria** (NewsMention PENDING/APPROVED/REJECTED) — verificado que a rota pública só retorna status=APPROVED. "Permissão no navegador" mapeada pro único ponto privilegiado do produto: a área de curadoria de notícias, protegida por token (não é um flag de UI sem correspondência — o token em si é a credencial, validada com timingSafeEqual no backend a cada request, fail-closed sem ADMIN_TOKEN configurado). IDOR percorrido em toda rota com parâmetro de ID nos dois backends (politicians/:id, news/admin/:id, deputies/{id}/similar) — nenhum recurso pertence a um usuário, e as ações privilegiadas estão atrás do guard. Chaves expostas checado nos três subprojetos (Node, Python, scripts) + Docker/env. XSS mapeado pra stack real (React/Vite): dangerouslySetInnerHTML, markdown; SQL injection checado como o equivalente de "input sem tratamento" no serviço Python (único lugar do projeto com SQL bruto em vez de ORM parametrizado).

1 achado — 1 Informativa — **nenhuma vulnerabilidade explorável encontrada**

Resumo executivo

Auditoria limpa. Apesar de ser o projeto mais complexo já auditado nesta série (2 backends — NestJS/Prisma e Python/FastAPI — mais 1 frontend e um cron de sincronização), as 5 categorias verificadas em código real não encontraram vulnerabilidade explorável. O único item registrado (AB1) é **informativo/housekeeping**: o mesmo componente shadcn morto encontrado em outros projetos deste autor. Pontos que mereciam atenção especial dado a complexidade — SQL bruto no serviço Python, CORS totalmente aberto, token de admin guardado no navegador — foram todos verificados individualmente e vieram corretos.



Pontos fortes (verificados, com evidência)

✓ **Fronteira de curadoria verificada de ponta a ponta** — GET /api/news/politicians/:id filtra explicitamente status: 'APPROVED' (news.service.ts) — menção pendente ou rejeitada nunca aparece na rota pública, mesmo com o id certo.

✓ **Token de admin é credencial real, não flag de UI** — AdminTokenGuard compara o header x-admin-token com crypto.timingSafeEqual (constant-time) contra ADMIN_TOKEN do ambiente, fail-closed (503) se a variável não existir. O frontend (NewsCuration.tsx) guarda o token em sessionStorage, nunca hardcoded — o "gate" da UI é só uma tela de login pro mesmo segredo que o backend exige de verdade.

✓ **Formulário de contato público falha fechado sem quebrar o resto** — ContactService lança ServiceUnavailableException se RESEND_API_KEY não estiver configurada — nunca finge sucesso silenciosamente (achado real documentado: o formulário antigo era decorativo e essa regressão foi corrigida).

✓ **Rate limit real na única rota pública de escrita** — @Throttle({ default: { limit: 5, ttl: 900000 } }) em POST /api/contact — 5 envios por 15 minutos, defesa efetiva contra abuso num endpoint sem auth/captcha por design.

✓ **CORS aberto é seguro dado o modelo de auth escolhido** — app.enableCors({ origin: '*' }) é decisão deliberada (API pública de leitura) — não abre CSRF nas rotas admin porque o token não é cookie/credencial ambiente: um site de terceiro não tem como fazer o navegador da vítima enviar x-admin-token automaticamente.

✓ **Nenhuma SQL concatenada com input do usuário** — A API Node usa só o query builder do Prisma (0 ocorrências de $queryRaw/$executeRaw). O único SQL bruto do projeto (serviço Python, psycopg2) usa

strings estáticas OU, no caso com f-string (clustering.py:98), os valores interpolados são politician_id vindos do próprio banco e forçados por int() antes — não há caminho de um valor de request chegar lá como string.

✓ **Parâmetros numéricos sempre limitados nas duas APIs** — FastAPI: k (2–20), n (1–50) validados via Query(ge=..., le=...) — Pydantic rejeita fora do intervalo antes do handler rodar. NestJS: ParseIntPipe em todo :id de rota.

✓ **Proxy de foto não aceita URL do cliente (sem SSRF)** — GET /api/politicians/:id/photo busca a URL só do campo photo_url já salvo no banco (photoUrlOf) — não existe rota de escrita pública pra esse campo; quem popula é só o sync-worker a partir da API oficial da Câmara/Senado.

✓ **Validação global de DTO com whitelist** — ValidationPipe({ transform: true, whitelist: true }) global no bootstrap — campo não esperado no body é descartado antes de chegar no service, em toda rota.

✓ **Scripts de sync/seed fora do runtime público** — sync-worker.ts e os scripts de scoring/seed rodam via cron, sem porta exposta nem rota de Traefik (confirmado no hetzner-infra) — não fazem parte da superfície de ataque HTTP.

Pontos fracos

Nenhum risco de segurança identificado. O item AB1 é hardening preventivo de código morto — vale fazer por higiene de repositório, não porque exista exploração hoje.

Achados detalhados

Informativa

AB1 · Inputs sem tratamento (equiv. XSS) / housekeeping

`src/components/ui/chart.tsx:77–95`

dangerouslySetInnerHTML em componente shadcn não utilizado

ChartStyle usa dangerouslySetInnerHTML pra gerar um <style> a partir de config.theme/config.color — o mesmo boilerplate padrão do shadcn/ui já encontrado em outros projetos deste autor (instalado via CLI, nunca importado de fato). Busca por <Chart em todo src/pages e src/components não encontrou nenhum uso. Sem exploração possível hoje: nenhum dado dinâmico do produto (scores, votos, notícias) alimenta esse config em lugar nenhum do código atual.

```
// src/components/ui/chart.tsx
<style
dangerouslySetInnerHTML={{
__html: Object.entries(THEMES).map(([theme, prefix]) => `
${prefix} [data-chart=${id}] { ... ${itemConfig.color} ... }
`).join('\n'),
}}
/>
// nunca importado: grep por "<Chart" em src/pages e src/components → 0 resultados
```

Impacto: Nenhum hoje. Se o componente Chart for adotado no futuro com config vindo de dado dinâmico (ex.: cor definida por partido ou por conteúdo vindo da API), o dangerouslySetInnerHTML atual injetaria isso sem sanitização numa tag <style> sem revisão adicional — é código morto com um padrão sensível embutido, não uma vulnerabilidade ativa.

Correção sugerida: Remover o componente Chart não utilizado (e o import do pacote recharts associado, se não usado em outro lugar) — reduz superfície de código que precisaria ser reauditado se algum dia for religado por engano.

Categorias sem achado (verificadas explicitamente)

✓ **1 — Isolamento de tenant / RLS** — Sem conta de usuário final — dado é público por design. Equivalente real (status APPROVED de NewsMention) verificado na rota pública de notícias.

✓ **2 — Permissão definida no navegador** — Único ponto privilegiado (curadoria) usa um token real validado no backend com timingSafeEqual — não é um flag de UI sem correspondência.

✓ **3 — IDOR** — Toda rota com :id revisada nos dois backends (politicians, news, deputies/similar). Nenhum recurso pertence a um usuário; ações privilegiadas estão atrás do AdminTokenGuard.

✓ **4 — Chaves expostas** — Nenhum hardcode nos três subprojetos. ADMIN_TOKEN e RESEND_API_KEY falham fechado (503) sem valor configurado; APIs governamentais (Câmara/Senado/TSE) são públicas, sem chave.

Recomendações priorizadas

Prior.	Ação	Por quê
P1	(opcional, baixo esforço) Remover componente shadcn Chart não utilizado (AB1)	chart.tsx nunca é importado — nenhuma fonte de dado dinâmico do produto o alimenta hoje, mas remover código morto com dangerouslySetInnerHTML é limpeza de baixo custo.

Sem P2/P3 — não há mais nenhuma ação de segurança pendente identificada nesta auditoria.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar. Só 1 issue aqui, de housekeeping — não há achado de segurança acionável que justifique uma issue de correção urgente.

--- ISSUE 1 ---

[Housekeeping] Remove componente shadcn Chart não utilizado (dangerouslySetInnerHTML morto)

Labels: security, severidade:informativa, housekeeping

```
**Problema**

`src/components/ui/chart.tsx:77-95` — `ChartStyle` usa `dangerouslySetInnerHTML` pra gerar um `<style>` a
partir de `config.theme`/`config.color`. Boilerplate padrão do shadcn/ui, instalado mas **nunca
importado** (`grep "<Chart"` em todo `src/pages` e `src/components` → 0 resultados).

**Por que vale remover**

Não é explorável hoje — nenhum dado dinâmico do produto (scores, votos, notícias, gastos) alimenta
`config` em lugar nenhum do código atual. Mas é código morto com um padrão sensível
(`dangerouslySetInnerHTML`) que passaria despercebido numa reintrodução futura.

**Evidência**

```tsx
<style
dangerouslySetInnerHTML={{
__html: Object.entries(THEMES).map(([theme, prefix]) => `...`).join('\n'),
}}
/>
```

Impacto

Nenhum hoje. Risco é só de reintrodução futura sem revisão.

Sugestão de correção

Remover `src/components/ui/chart.tsx` (e o import do pacote `recharts` associado, se não usado em outro
lugar) caso nenhum gráfico esteja planejado no roadmap próximo.

Critérios de aceite

- [ ] `chart.tsx` removido ou sua geração de CSS deixa de usar `dangerouslySetInnerHTML`.
- [ ] Build e testes do frontend continuam passando sem ele.
```

--- FIM ISSUE 1 ---